



Module Objective: In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py`.
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```
CLASS KnowledgeBase:
    // Attributes
    DECLARE facts AS Set           // To store unique string facts
    DECLARE rules AS List         // To store rules as Tuples:

    // Methods
    FUNCTION tell_fact(fact_string):
        ADD fact_string TO facts
```

```
FUNCTION tell_rule(premise_list, conclusion_string):  
    APPEND Tuple(premise_list, conclusion_string) TO rules  
  
FUNCTION clear_facts():  
    EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():  
    DECLARE new_facts_added = TRUE  
  
    WHILE new_facts_added == TRUE:  
        new_facts_added = FALSE  
  
        FOR EACH (premises, conclusion) IN rules:  
            IF conclusion IS NOT IN facts:  
  
                // Modus Ponens Check  
                IF ALL premises ARE IN facts:  
                    ADD conclusion TO facts  
                    new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In your main `agent.py`, instantiate the KB in the `__init__` method: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
Rule 1: `TargetVisible` $\wedge$ `HasDust` $\Rightarrow$ `SafeToEngage`
Rule 2: `SafeToEngage` $\wedge$ `BloodseekerMissing` $\Rightarrow$ `Retreat`

Step 3.2: Validating Feasibility in A*

Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
2. Before adding a neighbor to the `open_list`, clear the KB facts.
3. Feed the current percepts for that specific tile into the KB using `tell_fact()`.
4. Run `self.kb.forward_chain()`.
5. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly

mathematically before you attempt to integrate it into the visual grid.

```
from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce SafeToEngage"
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce Retreat"

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat"

    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()
```

Part 5: Theory-to-Implementation Mapping

Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

Reachability is a purely geometric test: a neighbour is "reachable" if it is inside the grid, is not a wall, and has not already been expanded - the normal A* open-list gating. Feasibility is a separate logical check layered on top of that geometry. In the neighbour-generation loop, once the physical reachability checks pass, I clear the Knowledge Base (`kb.clear_facts()`), feed that tile's sensor percepts into the KB with `tell_fact()` (`TargetVisible`, `HasDust`, `BloodseekerMissing`), and run `kb.forward_chain()`. Only if "Retreat" is NOT deduced do I admit the neighbour into the priority queue; if "Retreat" is deduced the tile is treated as Infeasible and skipped even though it is physically traversable. So the Knowledge Base enforces logical safety: reachability answers "can a path pass through here?", while feasibility answers "is it logically proven safe to step there?".

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

The `KnowledgeBase + forward_chain()` approach is declarative: the reasoning knowledge ("TargetVisible AND HasDust implies SafeToEngage") lives as data - a list of Horn-Clause rules - instead of imperative if/else branches scattered through the movement code. The inference is then applied uniformly by a single engine. If we hardcoded the checks we would write one if-statement per rule, and adding 50 new rules would mean writing and maintaining 50 new branches, worrying about rule ordering and cross-rule interactions, and bloating the agent code with game-specific logic. Declaratively, adding 50 rules is just 50 `tell_rule()` calls; the movement code stays $O(1)$ because it never changes - it just asks the KB whether the tile is feasible. The cost shifts from code complexity into a fixed, reusable engine.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally

known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

A Horn Clause is a rule of the shape "premises AND ... AND premiseN imply conclusion". My `forward_chain()` executes Modus Ponens - "if A is true and A implies B, then B is true" - as a loop. Each pass, for every (premises, conclusion) rule whose conclusion is not already a fact, it checks whether ALL premises are now present in the facts set; if so it adds the conclusion. Because adding one conclusion may satisfy another rule, a single pass is insufficient, so the while loop keeps iterating until a full pass derives no new facts, i.e. it computes the deductive closure of the rules over the percepts. Each iteration precisely replicates one Modus Ponens inference step over the premises present at that point.

Finalize and Lock Lab 05 Implementation



FACULTY OF COMPUTING